第 1 讲：程序执行、复杂度与从思路到代码

1. 本讲目标

完成本讲后，你应该能够：

- 按顺序模拟一段 Python 代码的执行；
- 区分变量、状态、循环和条件；
- 判断常见单层、双层和折半循环的复杂度；
- 理解时间复杂度不是精确运行时间；
- 把一个简单思路细化成可以编码的步骤；
- 发现空数组、负数、重复值等基本边界问题。

这一讲是后面所有数据结构的基础。你当前的主要困难不是完全没有思路，而是思路还没有具体到能够直接翻译为代码。

---

2. 程序是怎样执行的

计算机不会理解“找一下最大的数字”这种概括。它只能执行明确的小步骤。

观察下面的代码：

```
[python]
nums = [4, 1, 7, 3]
ans = nums[0]

for x in nums[1:]:
    if x > ans:
        ans = x

print(ans)
```

执行过程可以写成状态表：

当前 `x`	循环开始时 `ans`	`x > ans`	循环结束时 `ans`
1	4	False	4
7	4	True	7
3	7	False	7

这里的 `ans` 表示：

> 截至目前已经看过的元素中的最大值。

这句话叫作变量的“含义”，也可以理解为循环过程中始终想维持的事实。以后写算法时，先说清一个变量代表什么，代码会容易很多。

2.1 四个最基本的组成部分

许多简单算法都可以拆成：

1. **状态**：需要记住的信息；
2. **初始化**：开始时状态是什么；
3. **更新**：处理一个新元素后，状态如何变化；
4. **结果**：什么时候返回、返回什么。

对“找最大值”来说：

```
          ans
ans = nums[0]
  x > ans   ans = x
        ans
```

2.2 为什么 `ans = 0` 可能错误

下面的实现无法正确处理全部为负数的数组：

```
[python]
def find_max(nums):
    ans = 0
    for x in nums:
        if x > ans:
            ans = x
    return ans
```

例如：

```
[python]
nums = [-8, -3, -10]
```

函数会返回 `0`，但 `0` 根本不在数组中。

一个更合理的实现是：

```
[python]
def find_max(nums):
    if not nums:
        return None

    ans = nums[0]

    for x in nums[1:]:
        if x > ans:
            ans = x

    return ans
```

这里先单独处理空数组，因为空数组不存在 `nums[0]`。

注意：真实项目中也可以对空数组抛出异常。LeetCode 通常会在题目中规定输入是否可能为空。面试时如果题意没有说明，应主动确认。

---

3. 从自然语言到代码

“我知道怎么做”可能有不同程度。

3.1 不够具体的思路

还缺少的信息包括：

- 怎样判断以前见过这个元素？

- 需要保存什么？
- 找到后返回什么？
- 没找到时返回什么？

3. 2 可以编码的思路

```
seen
   x
x     seen      True
 x    seen
           False
```

3. 3 翻译为 Python

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)

    return False
```

这里值得注意两个位置：

1. 必须先检查 `x in seen`，再执行 `seen.add(x)`；
2. `return False` 必须在循环结束后，而不是循环内部。

错误示例：

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        seen.add(x)
        if x in seen:
            return True
```

这个版本永远会在第一个元素处返回 `True`，因为检查之前已经把当前元素放进了集合。

另一个错误示例：

```
[python]
def contains_duplicate(nums):
    seen = set()

    for x in nums:
        if x in seen:
            return True
        seen.add(x)
        return False
```

这个版本最多只检查第一个元素。第一次循环结束前就返回了 `False`。

3. 4 编码前的六个问题

对简单题，可以固定问：

```
1.
2.
3.
4.
```

5.
6.

如果这些问题都有明确答案，代码通常只是翻译。

---

4. 什么是时间复杂度

时间复杂度描述：

> 当输入规模 `n` 增大时，算法执行步骤的数量以什么速度增长。

它不是精确的秒数，也通常不计算每一个常数步骤。

4.1 `O(1)`：固定数量的操作

```
[python]
def first_element(nums):
    return nums[0]
```

无论 `nums` 有 10 个还是 10 万个元素，都只访问一次，因此是 `O(1)`。

`O(1)` 不一定只执行一行。只要执行次数不随着 `n` 增长，就可以看作 `O(1)`。

4.2 `O(n)`：遍历一次

```
[python]
def total(nums):
    ans = 0
    for x in nums:
        ans += x
    return ans
```

数组有 `n` 个元素，循环执行 `n` 次，所以是 `O(n)`。

下面仍然是 `O(n)`：

```
[python]
for x in nums:
    print(x)

for x in nums:
    print(x * 2)
```

总步骤大约是 `2n`。复杂度分析忽略常数倍，因此：

```
O(2n) = O(n)
```

4.3 `O(n²)`：对每个元素再次遍历

```
[python]
for x in nums:
    for y in nums:
        print(x, y)
```

外层执行 `n` 次，每次内层再执行 `n` 次，总计 `n × n`，所以是 `O(n²)`。

两数之和的暴力解法也是 `O(n²)`：

```
[python]
def two_sum_brute_force(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
```

虽然内层循环不是每次都完整执行 `n` 次，但比较总数仍然与 n^2 同一个增长等级。

4.4 嵌套循环不一定都是 $O(n^2)$

不要看到两个 `for` 就机械判断为 $O(n^2)$。

```
[python]
for row in matrix:
    for value in row:
        print(value)
```

如果整个矩阵一共有 N 个元素，每个元素只访问一次，那么复杂度相对于元素总数是 $O(N)$。

关键问题是：最内层语句总共执行了多少次？

4.5 $O(\log n)$：每次缩小一半

```
[python]
n = 100

while n > 1:
    n //= 2
```

变化过程：

```
100 → 50 → 25 → 12 → 6 → 3 → 1
```

每次都把问题规模缩小一半，因此循环次数大约是 $\log_2 n$，复杂度为 $O(\log n)$。

二分查找是最典型的 $O(\log n)$ 算法。

4.6 $O(n \log n)$

高效的通用排序通常是 $O(n \log n)$。现阶段先记住常见大小关系：

```
O(1) < O(log n) < O(n) < O(n log n) < O(n²)
```

当 $n = 100000$ 时，大致可以感受为：

复杂度	操作数量级
$O(1)$	固定少量
$O(\log n)$	约 17 次折半
$O(n)$	约 10 万
$O(n \log n)$	约 170 万
$O(n^2)$	约 100 亿

这不是精确运行时间，只用于感受增长速度。

---

5. 分析复杂度的基本规则

5.1 忽略常数

```
O(3n) → O(n)
O(100) → O(1)
```

5.2 只保留增长最快的项

```
O(n² + n + 10) → O(n²)
```

当 `n` 很大时，`n²` 起主导作用。

5.3 连续执行通常相加

```python
[python]
for x in nums:       # O(n)
    print(x)

for y in nums:       # O(n)
    print(y)
```

结果为 `O(n + n) = O(n)`。

5.4 真正嵌套通常相乘

```python
[python]
for x in nums:       # n
    for y in nums:   #    n
        print(x, y)
```

结果为 `O(n × n) = O(n²)`。

5.5 提前返回不改变最坏情况

```python
[python]
def contains(nums, target):
    for x in nums:
        if x == target:
            return True
    return False
```

如果目标恰好是第一个元素，执行一次即可。但最坏情况下目标不存在，需要检查所有元素，因此最坏时间复杂度是 `O(n)`。

面试中如果没有特别说明，通常分析最坏时间复杂度。

---

6. 什么是空间复杂度

空间复杂度描述算法运行时额外使用的空间怎样随输入规模增长。

6.1 `O(1)` 额外空间

```python
[python]
def total(nums):
    ans = 0
    for x in nums:
        ans += x
    return ans
```

不论数组多长，只使用少数变量，因此额外空间是 `O(1)`。

输入数组本身一般不算算法额外创建的空间。

6.2 `O(n)` 额外空间

```python
[python]
def copy_numbers(nums):
    result = []
    for x in nums:
        result.append(x)
    return result
```

`result` 的长度随着 `n` 增长，因此额外空间为 `O(n)`。

使用集合记录所有已见元素也可能需要 `O(n)` 空间：

```python
seen = set()
for x in nums:
    seen.add(x)
```

6.3 时间与空间的交换

判断重复元素可以用双重循环：

- 时间：`O(n²)`；
- 额外空间：`O(1)`。

也可以使用集合：

- 时间：平均 `O(n)`；
- 额外空间：`O(n)`。

这叫空间换时间。面试中经常需要说明这种取舍。

---

7. 常见边界条件

写完主逻辑后，至少检查：

7.1 空输入

```python
nums = []
```

是否访问了不存在的 `nums[0]`？

7.2 只有一个元素

```python
nums = [5]
```

循环是否正确？应该返回什么？

7.3 全部为负数

```python
nums = [-7, -2, -9]
```

最大值初始值是否错误地设成 `0`？

7.4 重复值

```python
nums = [3, 3]
```

题目要求返回值、下标还是布尔值？

7.5 答案在开头或结尾

边界位置容易暴露循环范围错误。

7.6 根本没有答案

循环结束后是否有明确返回？

---

8. 手动调试方法

如果代码结果不对，不要只靠反复修改。选一个最小反例，记录关键变量。

例如：

```
[python]
nums = [4, 2, 4]
```

对判断重复的算法记录：

当前 `x`	判断前 `seen`	是否已存在	操作
4	{}	False	加入 4
2	{4}	False	加入 2
4	{2, 4}	True	返回 True

手动跟踪能够帮助你发现：

- 更新顺序错误；
- `return` 缩进错误；
- 循环少执行或多执行一次；
- 状态含义和实际内容不一致。

---

9. 本讲必须记住的 Python 写法

遍历元素

```
[python]
for x in nums:
    print(x)
```

遍历下标

```
[python]
for i in range(len(nums)):
    print(i, nums[i])
```

同时获得下标和元素

```
[python]
for i, x in enumerate(nums):
    print(i, x)
```

创建和更新集合

```
[python]
seen = set()
seen.add(5)

if 5 in seen:
    print("    ")
```

提前返回

```
[python]
for x in nums:
    if x < 0:
        return True

return False
```

注意最后的 `return False` 位于循环外部。

---

10. 本讲总结

1. 计算机需要明确、逐步、可执行的指令。
2. 写代码前先确定状态、初始化、更新方式和返回条件。
3. 时间复杂度描述输入增大时操作数量的增长速度。
4. 单次遍历通常是 `O(n)`，真正的双重遍历通常是 `O(n²)`，每次折半通常是 `O(log n)`。
5. 空间复杂度关注额外使用的空间。
6. 能看懂解答不等于能够主动生成代码。
7. 练习时应从空白开始，并用小例子手动跟踪变量。

学完后，请关闭本讲，独立完成练习文件。不要先打开参考答案。